Lecture 31 — Computational Geometry I: Vectors, Orientation, Segments, Polygons, Convex Hull

> Year Curriculum · Prerequisites: Week 2 (STL, custom structs), Week 4 (sorting with comparators), Week 10 (gcd) · Estimated time: 2.5–3 hours

1. Overview

Geometry problems appear in almost every ICPC set, and they have a reputation for being "where teams lose time". The reason is rarely the mathematics; it is special cases (collinear points, vertical edges, touching segments, duplicate points) and floating-point error. This lecture builds a small, exact, integer-based library around one primitive — the **cross product** — and uses it to solve the standard tasks:

- orientation of three points,
- point-on-segment and segment-intersection tests,
- polygon area (shoelace) and lattice points (Pick's theorem),
- point-in-polygon,
- convex hull with Andrew's monotone chain.

Guiding rule: **if the input coordinates are integers, keep every predicate in integers.** Use floating point only when you must construct a new point (such as an intersection) or measure a distance. Regional lecture 8 and Nationals lecture 8 extend this library (point in convex polygon in O(log n), rotating calipers, half-plane intersection).

2. Points and vectors

We use one struct for both points and vectors (a point is the vector from the origin):

```
using ll = long long;
struct P {
    ll x, y;
    P operator+(P o) const { return {x + o.x, y + o.y}; }
    P operator-(P o) const { return {x - o.x, y - o.y}; }
    bool operator<(P o) const { return tie(x, y) < tie(o.x, o.y); }
    bool operator==(P o) const { return x == o.x && y == o.y; }
};
```

Coordinates up to 10^9 in absolute value give differences up to $2 \cdot 10^9$ and products up to $4 \cdot 10^{18}$, which still fits in a signed 64-bit integer (max $\approx 9.2 \cdot 10^{18}$). Beyond that, use `__int128` for the products.

2.1 Dot product

$a \cdot b = a_x b_x + a_y\, b_y = |a||b| \cos\theta$, where θ is the angle between them. Its sign tells whether the angle is acute (> 0), right ($= 0$), or obtuse (< 0). Also $|a|^2 = a \cdot a$, and the projection length of b onto a is $(a \cdot b)/|a|$.

2.2 Cross product

$a \times b = a_x b_y - a_y\, b_x = |a||b| \sin\theta$, where θ is the **signed** angle from a to b (counter-clockwise positive). Two facts make it the workhorse of 2D geometry:

1. **Sign = turn direction.** $a \times b > 0$ iff b is counter-clockwise from a (less than 180°), < 0 iff clockwise, $= 0$ iff parallel.
2. **Magnitude = area.** $|a \times b|$ is the area of the parallelogram spanned by a and b, so a triangle o, a, b has area $|(a - o) \times (b - o)|/2$.

2.3 Orientation

Define `cross(o, a, b) = (a − o) × (b − o)`. Walking o → a → b:

- `> 0`: left turn (counter-clockwise),
- `< 0`: right turn (clockwise),
- `= 0`: the three points are collinear.

CSES 2189 (Point Location Test) is exactly this: print LEFT, RIGHT or TOUCH for p_3 relative to the directed line $p_1 \to p_2$. With integer input the answer is exact; there is nothing to tune.

3. Segments

3.1 Point on segment

Point p lies on the closed segment [a, b] iff it is collinear with a and b *and* lies within their bounding box. The bounding-box check is valid only because collinearity has already been established. (An alternative to the box check is `dot(a − p, b − p) ≤ 0`: the vectors from p to the endpoints point in opposite directions, or one is zero.)

3.2 Segment intersection

Segments [a, b] and [c, d] intersect (share at least one point) iff one of the following holds:

1. **Proper crossing**: c and d are strictly on opposite sides of line ab, and a and b strictly on opposite sides of line cd. With d_i the signs of `cross(a,b,c)`, `cross(a,b,d)`, `cross(c,d,a)`, `cross(c,d,b)`: $d_1 \cdot d_2 < 0$ and $d_3 \cdot d_4 < 0$.
2. **Touching**: some endpoint of one segment lies on the other segment. This covers T-junctions, shared endpoints, collinear overlap, and degenerate segments that are single points.

Why is this complete? If no orientation is zero and the signs are not both opposite, then one segment lies entirely on one side of the other's line, so they cannot meet. If some orientation is zero, say c is collinear with ab, then either c is on [a, b] (caught by case 2), or c is outside it on the line; checking all four endpoint-on-segment conditions handles collinear overlaps, because two overlapping collinear segments always contain an endpoint of one inside the other.

Multiply the **signs**, not the raw cross products: two values of size 10^{18} multiplied overflow.

3.3 Intersection point of two lines

Write the lines as $p + t \cdot u$ and $q + s \cdot v$. If $u \times v = 0$ they are parallel (or identical). Otherwise, crossing both sides of $p + t \cdot u = q + s \cdot v$ with v gives $t = ((q - p) \times v)/(u \times v)$, and the point is $p + t \cdot u$. The numerator and denominator are exact integers; only the final division is floating point. If you need to know whether the intersection is a lattice point, check divisibility of the integer numerators instead of looking at the double.

3.4 Distance from a point to a line or segment

The distance from p to line ab is $|\text{cross}(a, b, p)|/|b - a|$ (parallelogram area divided by the base). For the segment, if the projection falls outside — `dot(p - a, b - a) < 0` or `dot(p - b, a - b) < 0` — the closest point is the nearer endpoint. When you only need to compare distances, compare **squared** distances to stay in integers.

4. Polygons

4.1 Shoelace formula

For a simple polygon with vertices $p_0, \ldots, p_{n-1}$ listed in order ($p_n = p_0$):

$2 \cdot \text{SignedArea} = \Sigma_i \, p_i \times p_{i+1}$.

The result is positive for counter-clockwise order and negative for clockwise, so take the absolute value for the area. Intuition: each term is twice the signed area of triangle (origin, p_i, p_{i+1}); parts of triangles outside the polygon are counted once positively and once negatively and cancel. The formula works for non-convex simple polygons.

Worked example, polygon (1,1), (4,2), (3,5), (1,4):

edge	cross
(1,1)→(4,2)	$1\cdot 2 - 1\cdot 4 = -2$
(4,2)→(3,5)	$4\cdot 5 - 2\cdot 3 = 14$
(3,5)→(1,4)	$3\cdot 4 - 5\cdot 1 = 7$
(1,4)→(1,1)	$1\cdot 1 - 4\cdot 1 = -3$

Sum 16, so the area is 8 and the order is counter-clockwise. CSES 2191 asks for 2·Area precisely so that the answer is an integer.

4.2 Pick's theorem

For a simple polygon whose vertices are lattice points, $A = I + B/2 - 1$, where I is the number of lattice points strictly inside and B the number on the boundary. An edge from (x_1, y_1) to (x_2, y_2) contains $\gcd(|x_2 - x_1|, |y_2 - y_1|)$ lattice points if we count one endpoint, so $B = \Sigma \gcd(|\Delta x|, |\Delta y|)$. Rearranged with the integer 2A: $I = (2A - B + 2)/2$. For the example: $B = 1 + 1 + 1 + 3 = 6$ and $I = (16 - 6 + 2)/2 = 6$ (CSES 2193).

4.3 Point in polygon

Ray casting: shoot a horizontal ray from p towards +x and count how many polygon edges it crosses; p is inside iff the count is odd. Implementation details that make it exact:

- First check whether p lies on an edge (boundary).
- Count edge (a, b) only if it straddles the horizontal line y = p.y using the half-open test `(a.y > p.y) != (b.y > p.y)`. A vertex exactly at height p.y then belongs to exactly one of its two edges, so rays passing through vertices are counted correctly, and horizontal edges are never counted.
- The crossing is to the right of p iff p is to the left of the upward-directed edge: for a.y < b.y check `cross(a, b, p) > 0`; for a.y > b.y check `cross(a, b, p) < 0`.

This is O(n) per query and works for any simple polygon. For convex polygons there is an O(log n) binary-search method (Regional 8).

5. Convex hull

The convex hull of a point set is the smallest convex polygon containing it. Its vertices are a subset of the points. **Andrew's monotone chain** algorithm:

1. Sort the points by x, then y, and remove duplicates.
2. **Lower hull**: scan left to right, maintaining a stack. Before pushing point p, pop the top while the last two stack points and p do *not* make a strict left turn (`cross(h[k−2], h[k−1], p) ≤ 0`).
3. **Upper hull**: the same scan from right to left, continuing on the same stack (but never popping into the lower hull).
4. The last point equals the first; drop it. The result is in counter-clockwise order, starting at the lexicographically smallest point.

Correctness sketch: after sorting, the leftmost and rightmost points are hull vertices. Between them, the lower hull is a sequence of left turns; any point that causes a non-left turn is inside (or on) the triangle formed with its neighbours and can never be a lower-hull vertex. Each point is pushed and popped at most once per chain, so the scan is O(n) and the sort dominates: **O(n log n)**.

Collinear points. With `≤ 0` the hull has no three collinear vertices (the minimal hull). Some problems (CSES 2195) want every input point that lies on the hull boundary; then pop only on a strict right turn (`< 0`). With that change, the case where *all* points are collinear must be handled separately (the two chains would each contain every point); return the sorted points in that case.

Degenerate inputs to always test: n = 1, n = 2, all points equal, all collinear, many points on one hull edge.

6. Precision

- With integer input, all the predicates above are exact. Do not introduce an epsilon.
- With real input or constructed points, compare using a tolerance: `x < -EPS`, `fabs(x) <= EPS`, `x > EPS`, where EPS is chosen relative to the magnitude of the values (1e−9 for coordinates around 10^3–10^4 is common).
- Prefer `long double` when doubles are needed; prefer `atan2(y, x)` over `atan(y/x)`; avoid `acos` of values that may be slightly outside [−1, 1] due to rounding (clamp first).
- Never use `==` on floating-point values.

7. Reference implementations

The following single library file solves CSES 2189, 2190, 2191, 2192, 2193 and 2195 depending on a task keyword in the first input token, so all of the functions above are tested together. In a contest you would paste only what you need.

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

struct P {
    ll x, y;
    P operator+(P o) const { return {x + o.x, y + o.y}; }
    P operator-(P o) const { return {x - o.x, y - o.y}; }
    bool operator<(P o) const { return tie(x, y) < tie(o.x, o.y); }
    bool operator==(P o) const { return x == o.x && y == o.y; }
};
ll dot(P a, P b) { return a.x * b.x + a.y * b.y; }
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
ll cross(P o, P a, P b) { return cross(a - o, b - o); }
int sgn(ll v) { return (v > 0) - (v < 0); }

bool onSegment(P a, P b, P p) {
    return cross(a, b, p) == 0 && min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
           min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
}

bool segIntersect(P a, P b, P c, P d) {
    int d1 = sgn(cross(a, b, c)), d2 = sgn(cross(a, b, d));
    int d3 = sgn(cross(c, d, a)), d4 = sgn(cross(c, d, b));
    if (d1 * d2 < 0 && d3 * d4 < 0) return true;
    return onSegment(a, b, c) || onSegment(a, b, d) || onSegment(c, d, a) || onSegment(c,
d, b);
}

ll area2(const vector<P>& p) {  // twice the signed area
    ll s = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) s += cross(p[i], p[(i + 1) % n]);
    return s;
}

int inPolygon(const vector<P>& poly, P p) {  // 0 outside, 1 inside, 2 boundary
    int n = poly.size();
    bool in = false;
    for (int i = 0; i < n; i++) {
        P a = poly[i], b = poly[(i + 1) % n];
        if (onSegment(a, b, p)) return 2;
        if ((a.y > p.y) != (b.y > p.y)) {
            ll c = cross(a, b, p);
            if (b.y > a.y ? c > 0 : c < 0) in = !in;
        }
    }
```

```cpp
    return in ? 1 : 0;
}

// Counter-clockwise hull. keepCollinear = true keeps points lying on hull edges.
vector<P> convexHull(vector<P> pts, bool keepCollinear) {
    sort(pts.begin(), pts.end());
    pts.erase(unique(pts.begin(), pts.end()), pts.end());
    int n = pts.size();
    if (n < 3) return pts;
    bool allCollinear = true;
    for (int i = 2; i < n && allCollinear; i++)
        if (cross(pts[0], pts[1], pts[i]) != 0) allCollinear = false;
    if (allCollinear) return keepCollinear ? pts : vector<P>{pts[0], pts[n - 1]};
    auto bad = [&](ll c) { return keepCollinear ? c < 0 : c <= 0; };
    vector<P> h(2 * n);
    int k = 0;
    for (int i = 0; i < n; i++) {
        while (k >= 2 && bad(cross(h[k - 2], h[k - 1], pts[i]))) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, lo = k + 1; i >= 0; i--) {
        while (k >= lo && bad(cross(h[k - 2], h[k - 1], pts[i]))) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    string task;
    cin >> task;
    auto readP = [&]() { P p; cin >> p.x >> p.y; return p; };
    if (task == "location") {            // CSES 2189
        int t; cin >> t;
        while (t--) {
            P a = readP(), b = readP(), c = readP();
            ll v = cross(a, b, c);
            cout << (v > 0 ? "LEFT" : v < 0 ? "RIGHT" : "TOUCH") << "\n";
        }
    } else if (task == "intersect") {    // CSES 2190
        int t; cin >> t;
        while (t--) {
            P a = readP(), b = readP(), c = readP(), d = readP();
            cout << (segIntersect(a, b, c, d) ? "YES" : "NO") << "\n";
        }
    } else if (task == "area") {         // CSES 2191 (prints 2 * area)
        int n; cin >> n;
        vector<P> p(n);
        for (auto& q : p) q = readP();
        cout << llabs(area2(p)) << "\n";
    } else if (task == "inside") {       // CSES 2192
        int n, m; cin >> n >> m;
        vector<P> p(n);
```

```
        for (auto& q : p) q = readP();
        const char* name[] = {"OUTSIDE", "INSIDE", "BOUNDARY"};
        while (m--) cout << name[inPolygon(p, readP())] << "\n";
    } else if (task == "pick") {         // CSES 2193
        int n; cin >> n;
        vector<P> p(n);
        for (auto& q : p) q = readP();
        ll A2 = llabs(area2(p)), B = 0;
        for (int i = 0; i < n; i++) {
            P d = p[(i + 1) % n] - p[i];
            B += __gcd(llabs(d.x), llabs(d.y));
        }
        cout << (A2 - B + 2) / 2 << " " << B << "\n";
    } else if (task == "hull") {         // CSES 2195 (boundary points included)
        int n; cin >> n;
        vector<P> p(n);
        for (auto& q : p) q = readP();
        vector<P> h = convexHull(p, true);
        cout << h.size() << "\n";
        for (P q : h) cout << q.x << " " << q.y << "\n";
    }
}
```

Sample checks (prefix each CSES sample with the task word):

- `area 4  1 1  4 2  3 5  1 4` → 16
- `pick 4  1 1  4 2  3 5  1 4` → 6 6
- `inside 4 3  1 1 4 2 3 5 1 4  2 3  3 1  1 3` → INSIDE, OUTSIDE, BOUNDARY
- `location 3  1 1 5 3 2 3  1 1 5 3 4 1  1 1 5 3 3 2` → LEFT, RIGHT, TOUCH
- `intersect 1  1 1 5 3 1 2 4 3` → NO; `intersect 1  1 1 5 3 1 1 4 3` → YES (shared endpoint)

The hull routine was additionally stress-tested against a brute-force $O(n^3)$ "is this point on the hull boundary?" checker on thousands of random small inputs, including many collinear points.

8. Common mistakes

1. **Overflow.** Use `long long` for coordinates and cross products; `int` silently overflows at $\sim 2 \cdot 10^9$. Do not multiply two cross products (use signs).
2. **Missing collinear cases** in segment intersection: two collinear overlapping segments, a segment that is a single point, shared endpoints.
3. **Bounding-box check without collinearity** in `onSegment` (or vice versa).
4. **Clockwise input.** The shoelace sum is negative; take the absolute value. Do not assume an input orientation unless the statement promises one.
5. **Ray casting through a vertex** counted twice (or zero times). The half-open straddle test fixes it.
6. **Hull edge cases**: duplicate points, $n \le 2$, all points collinear; `<=` vs `<` deciding whether collinear points are kept.

7. **Using epsilons on integer data.** Exact predicates are both simpler and correct.
8. **Floating-point equality.** Compare with tolerance, and be careful with `acos`, `sqrt` of slightly negative values.
9. **Forgetting the closing edge** ($p_{n-1} \to p_0$) in polygon loops; use `(i + 1) % n`.

9. Practice problems

- CSES 2189 — Point Location Test (https://cses.fi/problemset/task/2189)
- CSES 2190 — Line Segment Intersection (https://cses.fi/problemset/task/2190)
- CSES 2191 — Polygon Area (https://cses.fi/problemset/task/2191)
- CSES 2192 — Point in Polygon (https://cses.fi/problemset/task/2192)
- CSES 2193 — Polygon Lattice Points (https://cses.fi/problemset/task/2193)
- CSES 2195 — Convex Hull (https://cses.fi/problemset/task/2195)
- Kattis — polygonarea (https://open.kattis.com/problems/polygonarea)
- Kattis — segmentintersection (https://open.kattis.com/problems/segmentintersection) — needs the intersection point with doubles
- Kattis — pointinpolygon (https://open.kattis.com/problems/pointinpolygon)
- Kattis — convexhull (https://open.kattis.com/problems/convexhull)
- Codeforces 1C — Ancient Berland Circus (https://codeforces.com/problemset/problem/1/C)
- CSES 2194 — Minimum Euclidean Distance (https://cses.fi/problemset/task/2194) — stretch (closest pair, Nationals 8)

10. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, chapter 29 (geometry) and 30 (sweep line)
- cp-algorithms: "Basic Geometry", "Check if two segments intersect", "Oriented area of a triangle", "Convex Hull construction" (https://cp-algorithms.com/geometry/convex-hull.html)
- Victor Lecomte, *Handbook of geometry for competitive programmers* (free PDF) — excellent on precision and robust predicates
- KACTL geometry section (https://github.com/kth-competitive-programming/kactl) — a well-tested team-notebook library
- M. de Berg et al., *Computational Geometry: Algorithms and Applications*, chapter 1